

12.2.2 节练习

练习 12.26: 用 `allocator` 重写第 427 页中的程序。



12.3 使用标准库：文本查询程序

我们将实现一个简单的文本查询程序，作为标准库相关内容学习的总结。我们的程序允许用户在一个给定文件中查询单词。查询结果是单词在文件中出现的次数及其所在行的列表。如果一个单词在一行中出现多次，此行只列出一行。行会按照升序输出——即，第 7 行会在第 9 行之前显示，依此类推。

例如，我们可能读入一个包含本章内容（指英文版中的文本）的文件，在其中寻找单词 `element`。输出结果的前几行应该是这样的：

```
element occurs 112 times
(line 36) A set element contains only a key;
(line 158) operator creates a new element
(line 160) Regardless of whether the element
(line 168) When we fetch an element from a map, we
(line 214) If the element is not found, find returns
```

接下来还有大约 100 行，都是单词 `element` 出现的位置。



12.3.1 文本查询程序设计

485

开始一个程序的设计的一种好方法是列出程序的操作。了解需要哪些操作会帮助我们分析出需要什么样的数据结构。从需求入手，我们的文本查询程序需要完成如下任务：

- 当程序读取输入文件时，它必须记住单词出现的每一行。因此，程序需要逐行读取输入文件，并将每一行分解为独立的单词
- 当程序生成输出时，
 - 它必须能提取每个单词所关联的行号
 - 行号必须按升序出现且无重复
 - 它必须能打印给定行号中的文本。

利用多种标准库设施，我们可以很漂亮地实现这些要求：

- 我们将使用一个 `vector<string>` 来保存整个输入文件的一份拷贝。输入文件中的每行保存为 `vector` 中的一个元素。当需要打印一行时，可以用行号作为下标来提取行文本。
- 我们使用一个 `istringstream`（参见 8.3 节，第 287 页）来将每行分解为单词。
- 我们使用一个 `set` 来保存每个单词在输入文本中出现的行号。这保证了每行只出现一次且行号按升序保存。
- 我们使用一个 `map` 来将每个单词与它出现的行号 `set` 关联起来。这样我们就可以方便地提取任意单词的 `set`。

我们的解决方案还使用了 `shared_ptr`，原因稍后进行解释。

数据结构

虽然我们可以用 `vector`、`set` 和 `map` 来直接编写文本查询程序，但如果定义一个更

为抽象的解决方案，会更为有效。我们将从定义一个保存输入文件的类开始，这会令文件查询更为容易。我们将这个类命名为 `TextQuery`，它包含一个 `vector` 和一个 `map`。`vector` 用来保存输入文件的文本，`map` 用来关联每个单词和它出现的行号的 `set`。这个类将会有有一个用来读取给定输入文件的构造函数和一个执行查询的操作。

查询操作要完成的任务非常简单：查找 `map` 成员，检查给定单词是否出现。设计这个函数的难点是确定应该返回什么内容。一旦找到了一个单词，我们需要知道它出现了多少次、它出现的行号以及每行的文本。

返回所有这些内容的最简单的方法是定义另一个类，可以命名为 `QueryResult`，来保存查询结果。这个类会有一个 `print` 函数，完成结果打印工作。

在类之间共享数据

486

我们的 `QueryResult` 类要表达查询的结果。这些结果包括与给定单词关联的行号的 `set` 和这些行对应的文本。这些数据都保存在 `TextQuery` 类型的对象中。

由于 `QueryResult` 所需要的数据都保存在一个 `TextQuery` 对象中，我们就必须确定如何访问它们。我们可以拷贝行号的 `set`，但这样做可能很耗时。而且，我们当然不希望拷贝 `vector`，因为这可能会引起整个文件的拷贝，而目标只不过是为了打印文件的一小部分而已（通常会是这样）。

通过返回指向 `TextQuery` 对象内部的迭代器（或指针），我们可以避免拷贝操作。但是，这种方法开启了一个陷阱：如果 `TextQuery` 对象在对应的 `QueryResult` 对象之前被销毁，会发生什么？在此情况下，`QueryResult` 就将引用一个不再存在的对象中的数据。

对于 `QueryResult` 对象和对应的 `TextQuery` 对象的生存期应该同步这一观察结果，其实已经暗示了问题的解决方案。考虑到这两个类概念上“共享”了数据，可以使用 `shared_ptr`（参见 12.1.1 节，第 400 页）来反映数据结构中的这种共享关系。

使用 `TextQuery` 类

当我们设计一个类时，在真正实现成员之前先编写程序使用这个类，是一种非常有用的方法。通过这种方法，可以看到类是否具有我们所需要的操作。例如，下面的程序使用了 `TextQuery` 和 `QueryResult` 类。这个函数接受一个指向要处理的文件的 `ifstream`，并与用户交互，打印给定单词的查询结果

```
void runQueries(ifstream &infile)
{
    // infile 是一个 ifstream，指向我们要处理的文件
    TextQuery tq(infile); // 保存文件并建立查询 map
    // 与用户交互：提示用户输入要查询的单词，完成查询并打印结果
    while (true) {
        cout << "enter word to look for, or q to quit: ";
        string s;
        // 若遇到文件尾或用户输入了 'q' 时循环终止
        if (!(cin >> s) || s == "q") break;
        // 指向查询并打印结果
        print(cout, tq.query(s)) << endl;
    }
}
```

我们首先用给定的 `ifstream` 初始化一个名为 `tq` 的 `TextQuery` 对象。`TextQuery` 的构造函数读取输入文件，保存在 `vector` 中，并建立单词到所在行号的 `map`。

`while`（无限）循环提示用户输入一个要查询的单词，并打印出查询结果，如此往复。循环条件检测字面常量 `true`（参见 2.1.3 节，第 37 页），因此永远成功。循环的退出是通过 `if` 语句中的 `break`（参见 5.5.1 节，第 170 页）实现的。此 `if` 语句检查输入是否成功。如果成功，它再检查用户是否输入了 `q`。输入失败或用户输入了 `q` 都会使循环终止。一旦用户输入了要查询的单词，我们要求 `tq` 查找这个单词，然后调用 `print` 打印搜索结果。

12.3.1 节练习

练习 12.27: `TextQuery` 和 `QueryResult` 类只使用了我们已经介绍过的语言和标准库特性。不要提前看后续章节内容，只用已经学到的知识对这两个类编写你自己的版本。

练习 12.28: 编写程序实现文本查询，不要定义类来管理数据。你的程序应该接受一个文件，并与用户交互来查询单词。使用 `vector`、`map` 和 `set` 容器来保存来自文件的数据并生成查询结果。

练习 12.29: 我们曾经用 `do while` 循环来编写管理用户交互的循环（参见 5.4.4 节，第 169 页）。用 `do while` 重写本节程序，解释你倾向于哪个版本，为什么。



12.3.2 文本查询程序类的定义

我们以 `TextQuery` 类的定义开始。用户创建此类的对象时会提供一个 `istream`，用来读取输入文件。这个类还提供一个 `query` 操作，接受一个 `string`，返回一个 `QueryResult` 表示 `string` 出现的那些行。

设计类的数据成员时，需要考虑与 `QueryResult` 对象共享数据的需求。`QueryResult` 类需要共享保存输入文件的 `vector` 和保存单词关联的行号的 `set`。因此，这个类应该有两个数据成员：一个指向动态分配的 `vector`（保存输入文件）的 `shared_ptr` 和一个 `string` 到 `shared_ptr<set>` 的 `map`。`map` 将文件中每个单词关联到一个动态分配的 `set` 上，而此 `set` 保存了该单词所出现的行号。

为了使代码更易读，我们还会定义一个类型成员（参见 7.3.1 节，第 243 页）来引用行号，即 `string` 的 `vector` 中的下标：

```
class QueryResult; // 为了定义函数 query 的返回类型，这个定义是必需的
class TextQuery {
public:
    using line_no = std::vector<std::string>::size_type;
    TextQuery(std::ifstream&);
    QueryResult query(const std::string&) const;
private:
    std::shared_ptr<std::vector<std::string>> file; // 输入文件
    // 每个单词到它所在的行号的集合的映射
    std::map<std::string,
            std::shared_ptr<std::set<line_no>>> wm;
};
```

这个类定义最困难的部分是解开类名。与往常一样，对于可能置于头文件中的代码，在使用标准库名字时要加上 `std::`（参见 3.1 节，第 74 页）。在本例中，我们反复使用了 `std::`，

使得代码开始可能有些难读。例如，

```
std::map<std::string, std::shared_ptr<std::set<line_no>>> wm;
```

如果写成下面的形式可能就更好理解一些

```
map<string, shared_ptr<set<line_no>>> wm;
```

TextQuery 构造函数

TextQuery 的构造函数接受一个 ifstream，逐行读取输入文件：

// 读取输入文件并建立单词到行号的映射

```
TextQuery::TextQuery(ifstream &is): file(new vector<string>)
{
    string text;
    while (getline(is, text)) {           // 对文件中每一行
        file->push_back(text);           // 保存此行文本
        int n = file->size() - 1;         // 当前行号
        istringstream line(text);         // 将行文本分解为单词
        string word;
        while (line >> word) {           // 对行中每个单词
            // 如果单词不在 wm 中，以之为下标在 wm 中添加一项
            auto &lines = wm[word];       // lines 是一个 shared_ptr
            if (!lines) // 在我们第一次遇到这个单词时，此指针为空
                lines.reset(new set<line_no>); // 分配一个新的 set
            lines->insert(n);              // 将此行号插入 set 中
        }
    }
}
```

构造函数的初始化器分配一个新的 vector 来保存输入文件中的文本。我们用 getline 逐行读取输入文件，并存入 vector 中。由于 file 是一个 shared_ptr，我们用->运算符解引用 file 来提取 file 指向的 vector 对象的 push_back 成员。

接下来我们用 一个 istringstream（参见 8.3 节，第 287 页）来处理刚刚读入的一行中的每个单词。内层 while 循环用 istringstream 的输入运算符来从当前行读取每个单词，存入 word 中。在 while 循环内，我们用 map 下标运算符提取与 word 相关联的 shared_ptr<set>，并将 lines 绑定到此指针。注意，lines 是一个引用，因此改变 lines 也会改变 wm 中的元素。

若 word 不在 map 中，下标运算符会将 word 添加到 wm 中（参见 11.3.4 节，第 387 页），与 word 关联的值进行值初始化。这意味着，如果下标运算符将 word 添加到 wm 中，lines 将是一个空指针。如果 lines 为空，我们分配一个新的 set，并调用 reset 更新 lines 引用的 shared_ptr，使其指向这个新分配的 set。

不管是否创建了一个新的 set，我们都调用 insert 将当前行号添加到 set 中。由于 lines 是一个引用，对 insert 的调用会将新元素添加到 wm 中的 set 中。如果一个给定单词在同一行中出现多次，对 insert 的调用什么都不会做。

◀ 489

QueryResult 类

QueryResult 类有三个数据成员：一个 string，保存查询单词；一个 shared_ptr，指向保存输入文件的 vector；一个 shared_ptr，指向保存单词出现行号的 set。它唯

一的一个成员函数是一个构造函数，初始化这三个数据成员：

```
class QueryResult {
friend std::ostream& print(std::ostream&, const QueryResult&);
public:
    QueryResult(std::string s,
                std::shared_ptr<std::set<line_no>> p,
                std::shared_ptr<std::vector<std::string>> f):
        sought(s), lines(p), file(f) { }
private:
    std::string sought; // 查询单词
    std::shared_ptr<std::set<line_no>> lines;           // 出现的行号
    std::shared_ptr<std::vector<std::string>> file;     // 输入文件
};
```

构造函数的唯一工作是将参数保存在对应的数据成员中，这是在其初始化器列表中完成的（参见 7.1.4 节，第 237 页）。

query 函数

query 函数接受一个 string 参数，即查询单词，query 用它来在 map 中定位对应的行号 set。如果找到了这个 string，query 函数构造一个 QueryResult，保存给定 string、TextQuery 的 file 成员以及从 wm 中提取的 set。

唯一的问题是：如果给定 string 未找到，我们应该返回什么？在这种情况下，没有可返回的 set。为了解决此问题，我们定义了一个局部 static 对象，它是一个指向空的行号 set 的 shared_ptr。当未找到给定单词时，我们返回此对象的一个拷贝：

```
QueryResult
TextQuery::query(const string &sought) const
{
    // 如果未找到 sought，我们将返回一个指向此 set 的指针
    static shared_ptr<set<line_no>> nodata(new set<line_no>);
    // 使用 find 而不是下标运算符来查找单词，避免将单词添加到 wm 中！
    auto loc = wm.find(sought);
    if (loc == wm.end())
        return QueryResult(sought, nodata, file); // 未找到
    else
        return QueryResult(sought, loc->second, file);
}
```

490 打印结果

print 函数在给定的流上打印出给定的 QueryResult 对象：

```
ostream &print(ostream & os, const QueryResult &qr)
{
    // 如果找到了单词，打印出现次数和所有出现的位置
    os << qr.sought << " occurs " << qr.lines->size() << " "
        << make_plural(qr.lines->size(), "time", "s") << endl;
    // 打印单词出现的每一行
    for (auto num : *qr.lines) // 对 set 中每个单词
        // 避免行号从 0 开始给用户带来的困惑
        os << "\t(line " << num + 1 << " ) "
```

```
        << *(qr.file->begin() + num) << endl;  
    return os;  
}
```

我们调用 `qr.lines` 指向的 `set` 的 `size` 成员来报告单词出现了多少次。由于 `set` 是一个 `shared_ptr`，必须解引用 `lines`。调用 `make_plural`（参见 6.3.2 节，第 201 页）来根据大小是否等于 1 打印 `time` 或 `times`。

在 `for` 循环中，我们遍历 `lines` 所指向的 `set`。`for` 循环体打印行号，并按人们习惯的方式调整计数值。`set` 中的数值就是 `vector` 中元素的下标，从 0 开始编号。但大多数用户认为第一行的行号应该是 1，因此我们对每个行号都加上 1，转换为人们更习惯的形式。

我们用行号从 `file` 指向的 `vector` 中提取一行文本。回忆一下，当给一个迭代器加上一个数时，会得到 `vector` 中相应偏移之后位置的元素（参见 3.4.2 节，第 99 页）。因此，`file->begin()+num` 即为 `file` 指向的 `vector` 中第 `num` 个位置的元素。

注意此函数能正确处理未找到单词的情况。在此情况下，`set` 为空。第一条输出语句会注意到单词出现了 0 次。由于 `*res.lines` 为空，`for` 循环一次也不会执行。

12.3.2 节练习

练习 12.30: 定义你自己版本的 `TextQuery` 和 `QueryResult` 类，并执行 12.3.1 节（第 431 页）中的 `runQueries` 函数。

练习 12.31: 如果用 `vector` 代替 `set` 保存行号，会有什么差别？哪种方法更好？为什么？

练习 12.32: 重写 `TextQuery` 和 `QueryResult` 类，用 `StrBlob` 代替 `vector<string>` 保存输入文件。

练习 12.33: 在第 15 章中我们将扩展查询系统，在 `QueryResult` 类中将会需要一些额外的成员。添加名为 `begin` 和 `end` 的成员，返回一个迭代器，指向一个给定查询返回的行号的 `set` 中的位置。再添加一个名为 `get_file` 的成员，返回一个 `shared_ptr`，指向 `QueryResult` 对象中的文件。

小结

在 C++ 中，内存是通过 `new` 表达式分配，通过 `delete` 表达式释放的。标准库还定义了一个 `allocator` 类来分配动态内存块。

分配动态内存的程序应负责释放它所分配的内存。内存的正确释放是非常容易出错的地方：要么内存永远不会被释放，要么在仍有指针引用它时就被释放了。新的标准库定义了智能指针类型——`shared_ptr`、`unique_ptr` 和 `weak_ptr`，可令动态内存管理更为安全。对于一块内存，当没有任何用户使用它时，智能指针会自动释放它。现代 C++ 程序应尽可能使用智能指针。

术语表

allocator 标准库类，用来分配未构造的内存。

空悬指针 (dangling pointer) 一个指针，指向曾经保存一个对象但现在已释放的内存。众所周知，空悬指针引起的程序错误非常难以调试。

delete 释放 `new` 分配的内存。`delete p` 释放对象，`delete []p` 释放 `p` 指向的数组。`p` 可以为空，或者指向 `new` 分配的内存。

释放器 (deleter) 传递给智能指针的函数，用来代替 `delete` 释放指针绑定的对象。

析构函数 (destructor) 特殊的成员函数，负责在对象离开作用域或被释放时完成清理工作。

动态分配的 (dynamically allocated) 在自由空间中分配的对象。在自由空间中分配的对象直到被显式释放或程序结束才会销毁。

自由空间 (free store) 程序可用的内存池，保存动态分配的对象。

堆 (heap) 自由空间的同义词。

new 从自由空间分配内存。`new T` 分配并构造一个类型为 `T` 的对象，并返回一个指向该对象的指针。如果 `T` 是一个数组类型，`new` 返回一个指向数组首元素的指针。类似的，`new [n] T` 分配 `n` 个类型为 `T` 的对象，并返回指向数组首元素的指针。

默认情况下，分配的对象进行默认初始化。我们也可以提供可选的初始化器。

定位 new (placement new) 一种 `new` 表达式形式，接受一些额外的参数，在 `new` 关键字后面的括号中给出。例如，`new (nothrow) int` 告诉 `new` 不要抛出异常。

引用计数 (reference count) 一个计数器，记录有多少用户共享一个对象。智能指针用它来判断什么时候释放所指向的对象是安全的。

shared_ptr 提供所有权共享的智能指针：对共享对象来说，当最后一个指向它的 `shared_ptr` 被销毁时会被释放。

智能指针 (smart pointer) 标准库类型，行为类似指针，但可以检查什么时候使用指针是安全的。智能指针类型负责在恰当的时候释放内存。

unique_ptr 提供独享所有权的智能指针：当 `unique_ptr` 被销毁时，它指向的对象被释放。`unique_ptr` 不能直接拷贝或赋值。

weak_ptr 一种智能指针，指向由 `shared_ptr` 管理的对象。在确定是否应释放对象时，`shared_ptr` 并不把 `weak_ptr` 统计在内。